

Prompt: Create a C program that reads a text file line by line executes the commands on each line to prioritize emails. The emails need to be stored in a list-based Max heap. The emails are prioritized from most important sender to least important sender: Boss, Subordinate, Peer, ImportantPerson, OtherPerson. The user will input a text file with each line having a list of either four commands: EMAIL (which is followed by the sender category, subject line, and date, each separated only by commas except the EMAIL command which is separated by a space)(The EMAIL command adds the email and the extra information into the Max Heap), NEXT (which prints the newest most important email's information), READ (which removes the newest most important email from the Max Heap, adjusts the Max Heap accordingly, and prints a message saying the CEO read it), and COUNT (Which prints how many emails are in the Max Heap to still be read). The text file's commands are separated by new lines. The EMAIL command has the sender category, subject line, and date on the same line as the EMAIL text command. Read and perform the commands in the text file line-by-line.

Gemini Code (prompted online on personal computer (gemini pro)):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_LINE_LEN 512
#define MAX_STR_LEN 128

// Enum representing sender priority (higher values indicate higher priority)
typedef enum {
    OTHER_PERSON = 0,
    IMPORTANT_PERSON = 1,
    PEER = 2,
    SUBORDINATE = 3,
    BOSS = 4,
    UNKNOWN = -1
} Priority;

// Structure to store Email information
typedef struct {
    Priority priority;
    char category[MAX_STR_LEN];
    char subject[MAX_STR_LEN];
    char date[MAX_STR_LEN];
    unsigned long long id; // Tracks arrival order to prioritize the newest on ties
} Email;

// Max Heap Structure using a dynamic array (list-based representation)
typedef struct {
```

```

    Email *arr;
    int size;
    int capacity;
} MaxHeap;

// Global counter tracking insertion age
unsigned long long insertion_counter = 0;

// Helper to map string category to Priority enum
Priority get_priority(const char *category) {
    if (strcmp(category, "Boss") == 0) return BOSS;
    if (strcmp(category, "Subordinate") == 0) return SUBORDINATE;
    if (strcmp(category, "Peer") == 0) return PEER;
    if (strcmp(category, "ImportantPerson") == 0) return IMPORTANT_PERSON;
    if (strcmp(category, "OtherPerson") == 0) return OTHER_PERSON;
    return UNKNOWN;
}

// Comparison function: returns 1 if email 'a' has higher max-heap precedence than 'b'
int has_higher_priority(Email a, Email b) {
    if (a.priority > b.priority) return 1;
    if (a.priority < b.priority) return 0;
    // Tie-breaker: If priority is identical, the newest email (higher id) comes first
    return (a.id > b.id) ? 1 : 0;
}

// Swap two email objects in the heap
void swap(Email *x, Email *y) {
    Email temp = *x;
    *x = *y;
    *y = temp;
}

// Create and initialize the heap
MaxHeap* create_heap(int capacity) {
    MaxHeap *heap = (MaxHeap*)malloc(sizeof(MaxHeap));
    heap->capacity = capacity;
    heap->size = 0;
    heap->arr = (Email*)malloc(capacity * sizeof(Email));
    return heap;
}

// Shift up to maintain heap property
void heapify_up(MaxHeap *heap, int index) {

```

```

while (index > 0) {
    int parent = (index - 1) / 2;
    if (has_higher_priority(heap->arr[index], heap->arr[parent])) {
        swap(&heap->arr[index], &heap->arr[parent]);
        index = parent;
    } else {
        break;
    }
}
}

// Shift down to maintain heap property
void heapify_down(MaxHeap *heap, int index) {
    int highest = index;
    int left = 2 * index + 1;
    int right = 2 * index + 2;

    if (left < heap->size && has_higher_priority(heap->arr[left], heap->arr[highest])) {
        highest = left;
    }
    if (right < heap->size && has_higher_priority(heap->arr[right], heap->arr[highest])) {
        highest = right;
    }
    if (highest != index) {
        swap(&heap->arr[index], &heap->arr[highest]);
        heapify_down(heap, highest);
    }
}

// Insert a new email item
void insert(MaxHeap *heap, Email email) {
    if (heap->size == heap->capacity) {
        heap->capacity *= 2;
        heap->arr = (Email*)realloc(heap->arr, heap->capacity * sizeof(Email));
    }
    heap->arr[heap->size] = email;
    heapify_up(heap, heap->size);
    heap->size++;
}

// Remove and return the maximum priority item
Email extract_max(MaxHeap *heap) {
    Email max_email = heap->arr[0];
    heap->arr[0] = heap->arr[heap->size - 1];

```

```

    heap->size--;
    if (heap->size > 0) {
        heapify_down(heap, 0);
    }
    return max_email;
}

```

```

// Deallocate heap arrays
void free_heap(MaxHeap *heap) {
    free(heap->arr);
    free(heap);
}

```

```

// Safely clean newline and carriage return characters from end of lines
void trim_newline(char *str) {
    int len = strlen(str);
    while (len > 0 && (str[len - 1] == '\n' || str[len - 1] == '\r')) {
        str[len - 1] = '\0';
        len--;
    }
}

```

```

int main() {
    char filename[MAX_STR_LEN];
    printf("Enter input file name: ");
    if (scanf("%127s", filename) != 1) return 1;

    FILE *file = fopen(filename, "r");
    if (!file) {
        printf("Error: Could not open file %s\n", filename);
        return 1;
    }

```

```

    MaxHeap *heap = create_heap(10);
    char line[MAX_LINE_LEN];

```

```

    while (fgets(line, sizeof(line), file)) {
        trim_newline(line);
        if (strlen(line) == 0) continue;

```

```

        // Separate command text from arguments by targeting the first space delimiter
        char line_copy[MAX_LINE_LEN];
        strcpy(line_copy, line);

```

```

char *cmd = strtok(line_copy, " ");
if (cmd == NULL) continue;

if (strcmp(cmd, "EMAIL") == 0) {
    // Find the start of arguments past the command name text and its space
    char *args_start = line + 6;

    char args_copy[MAX_LINE_LEN];
    strcpy(args_copy, args_start);

    // Tokenize strict comma boundaries (spaces inside subject fields are left intact)
    char *cat = strtok(args_copy, ",");
    char *subj = strtok(NULL, ",");
    char *dt = strtok(NULL, ",");

    if (cat && subj && dt) {
        Email new_email;
        new_email.id = ++insertion_counter;

        strcpy(new_email.category, cat);
        strcpy(new_email.subject, subj);
        strcpy(new_email.date, dt);
        new_email.priority = get_priority(cat);

        insert(heap, new_email);
    }
}
else if (strcmp(cmd, "NEXT") == 0) {
    if (heap->size > 0) {
        Email top = heap->arr[0];
        printf("[%s] Subject: %s | Date: %s\n", top.category, top.subject, top.date);
    } else {
        printf("No emails in queue.\n");
    }
}
else if (strcmp(cmd, "READ") == 0) {
    if (heap->size > 0) {
        Email read_email = extract_max(heap);
        printf("The CEO read it: [%s] %s\n", read_email.category, read_email.subject);
    } else {
        printf("No emails to read.\n");
    }
}
else if (strcmp(cmd, "COUNT") == 0) {

```

```

        printf("%d\n", heap->size);
    }
}

fclose(file);
free_heap(heap);
return 0;
}

```

Copilot's code (also prompted online on computer):

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_LINE 512
#define MAX_HEAP 1000

// Priority mapping
int getPriority(const char *sender) {
    if (strcmp(sender, "Boss") == 0) return 5;
    if (strcmp(sender, "Subordinate") == 0) return 4;
    if (strcmp(sender, "Peer") == 0) return 3;
    if (strcmp(sender, "ImportantPerson") == 0) return 2;
    if (strcmp(sender, "OtherPerson") == 0) return 1;
    return 0;
}

// Email structure
typedef struct {
    char sender[64];
    char subject[128];
    char date[64];
    int priority;
} Email;

// Max Heap
Email heap[MAX_HEAP];
int heapSize = 0;

// Swap helper
void swap(Email *a, Email *b) {

```

```

    Email temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify up
void heapifyUp(int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (heap[index].priority > heap[parent].priority) {
            swap(&heap[index], &heap[parent]);
            index = parent;
        } else break;
    }
}

// Heapify down
void heapifyDown(int index) {
    while (1) {
        int left = index * 2 + 1;
        int right = index * 2 + 2;
        int largest = index;

        if (left < heapSize && heap[left].priority > heap[largest].priority)
            largest = left;
        if (right < heapSize && heap[right].priority > heap[largest].priority)
            largest = right;

        if (largest != index) {
            swap(&heap[index], &heap[largest]);
            index = largest;
        } else break;
    }
}

// Add email
void addEmail(const char *sender, const char *subject, const char *date) {
    Email e;
    strcpy(e.sender, sender);
    strcpy(e.subject, subject);
    strcpy(e.date, date);
    e.priority = getPriority(sender);

    heap[heapSize] = e;
}

```

```

    heapifyUp(heapSize);
    heapSize++;
}

// Print newest most important email
void nextEmail() {
    if (heapSize == 0) {
        printf("No emails to show.\n");
        return;
    }
    Email *e = &heap[0];
    printf("Next Email -> Sender: %s | Subject: %s | Date: %s\n",
        e->sender, e->subject, e->date);
}

// Read (remove) newest most important email
void readEmail() {
    if (heapSize == 0) {
        printf("No emails to read.\n");
        return;
    }
    Email e = heap[0];

    // Replace root with last element
    heap[0] = heap[heapSize - 1];
    heapSize--;
    heapifyDown(0);

    printf("CEO read email from %s about \"%s\" dated %s.\n",
        e.sender, e.subject, e.date);
}

// Count emails
void countEmails() {
    printf("Emails remaining: %d\n", heapSize);
}

// Parse EMAIL command
void parseEmailCommand(char *line) {
    // Format: EMAIL Sender,Subject,Date
    char *cmd = strtok(line, " ");
    char *data = strtok(NULL, "\n");

    if (!data) return;

```

```

char *sender = strtok(data, ",");
char *subject = strtok(NULL, ",");
char *date = strtok(NULL, ",");

if (sender && subject && date) {
    addEmail(sender, subject, date);
}
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: %s <commands_file>\n", argv[0]);
        return 1;
    }

    FILE *fp = fopen(argv[1], "r");
    if (!fp) {
        printf("Could not open file.\n");
        return 1;
    }

    char line[MAX_LINE];

    while (fgets(line, sizeof(line), fp)) {
        // Remove newline
        line[strcspn(line, "\n")] = 0;

        if (strncmp(line, "EMAIL", 5) == 0) {
            parseEmailCommand(line);
        }
        else if (strcmp(line, "NEXT") == 0) {
            nextEmail();
        }
        else if (strcmp(line, "READ") == 0) {
            readEmail();
        }
        else if (strcmp(line, "COUNT") == 0) {
            countEmails();
        }
    }

    fclose(fp);
    return 0;
}

```

}

COMPARISON:

Notable comparisons (code structure)

	Gemini	Copilot
String lengths	Gemini uses one main variable for the email string lengths, which is MAX_STR_LEN. This variable is used multiple times.	Copilot simply uses set numbers instead of a variable. This might be more difficult to maintain (for example, if the importance of emails ever needed to be changed, its easier to change when the values are assigned to variables).
priority	Gemini creates an enum group, defining each category as a set number. Has a separate function that reads the string in the email to assign one of those priority categories to it.	Copilot uses a function with multiple if-statements to return a set number if one of the categories is used.
Heapify down	Gemini uses recursion to heapify down. This increases space complexity .	Copilot uses a while loop to heapify down, consistently changin a variabl that instructs where to use the code until the break line is used.
main	Gemini's main is longer. I believe some of what copilot does in functions, gemini gets done in the main. I also believe gemini does a better job handling cases	Copilot's main is smaller and seems to focus on calling whatever functions are necessary for each line. It also seems to take arguments inside of main, which im not sure have a source yet.
comments	Gemini gives fairly helpful comments. Its not a lot, but makes the code easier to follow.	Copilot has very little comments, and is much harder to follow. Most examples of code simply says what happens, and comments of functions share the same words (e. g. //parse EMAIL command, followed by void parseEmailCommand())
SPACE	Gemini's space complexity depends on the amount of emails input, so itll be O(n). BUT, gemini uses recursion which could add	Copilot's space complexity is just as gemini's so itll be O(n).

	Olog(n) to the mix, its not as much, but its my main idea into proving its complexity.	
TIME	The main time complexities involve heapify down. This speed is better than moving down each line in the list (which would make it O(n)). The biggest deal is the heapify up and down, which has a time complexity of (log(n)), and the amount of commands.	Copilot's time complexity is near identical to Gemini's.

CHOICE: Gemini prompt

Mainly for the fact that gemini's code has more beneficial structures than copilot and makes it easier to follow. Since it is a trend that copilot seems to be more confusing in structure and generally does less to maintain, i will most likely look for a different generator to use next time.

- Modifying code (what do i plan on changing?)

- The main item to change was the recursive function in the code. Since it costs more space, going down copilot's route of using an iterative function would help its execution time.
-
- Comments - a better idea of what each function and block of code does is essential in keeping maintainability. Gemini's prompt gives a good start, but could always be more descriptive.
-
- Looking into the code, the identified potential issues (like using two reads in a row or next and read when there are no more emails in the code) are handled, so we can confirm after checking the code.

CHANGED CODE:

```
//
//Name of program: 3228668_Warren_assignment2.c
//Discription: A program that takes in a list of commands holding emails,
//adds them to a max heap by priority, and completes each list of
commands.
//commands able to be used:
//EMAIL (category, subject, date)
//READ
```

```

//COUNT
//NEXT
//
//Inputs: .txt file where each line contains only one of any of the
above mentioned commands.
// Output: all commands completed (printed statements that commands were
done)
// collaborators: none
//other sources: Google Gemini's code
//Author: Christopher Warren
//Creation date: 9/17/2026
// Revision date: 9/17/2026
// Revisions: added comments, altered heapify-up to be iterative

#include <stdio.h> //input/output header file
#include <stdlib.h> //library file
#include <string.h> // header file to manipulate strings

#define MAX_LINE_LEN 512 //this is for keeping memory for the command file
#define MAX_STR_LEN 128 //this is for keeping memory for the emails

// Enum representing sender priority (higher values indicate higher
priority)
//the boss is the most important, hile other people are the least,
// and unknown are even lower.
typedef enum {
    OTHER_PERSON = 0,
    IMPORTANT_PERSON = 1,
    PEER = 2,
    SUBORDINATE = 3,
    BOSS = 4,
    UNKNOWN = -1
} Priority; //type name Priority

// Structure to store Email information
typedef struct {
    Priority priority;
    char category[MAX_STR_LEN]; //e. g. "boss"
    char subject[MAX_STR_LEN]; //e. g. "Quarterly Report"
    char date[MAX_STR_LEN]; //e. g. "9/17/2026"

```

```

    unsigned long long id; // Tracks arrival order to prioritize the
newest on ties
} Email; //type name Email

// Max Heap Structure using a dynamic array (list-based representation)
typedef struct {
    Email *arr; //declares a pointer for emails
    int size;
    int capacity;
} MaxHeap; //type name MaxHep

// Global counter tracking insertion age (in case multiple emails have
same priority)
unsigned long long insertion_counter = 0;

// Helper to map string category to Priority enum
// Inputs: category (the second part of the EMAIL command line)
// Output: the correct Priority value
//strcmp() checks if theyre identical or not (0 means it is)
Priority get_priority(const char *category) {
    if (strcmp(category, "Boss") == 0) return BOSS;
    if (strcmp(category, "Subordinate") == 0) return SUBORDINATE;
    if (strcmp(category, "Peer") == 0) return PEER;
    if (strcmp(category, "ImportantPerson") == 0) return IMPORTANT_PERSON;
    if (strcmp(category, "OtherPerson") == 0) return OTHER_PERSON;
    return UNKNOWN;
}

// Comparison function:
// Input: two emails (email a and email b)
//Output: returns 1 if email 'a' has higher max-heap precedence than 'b'
int has_higher_priority(Email a, Email b) {
    if (a.priority > b.priority) return 1;
    if (a.priority < b.priority) return 0;
    // Tie-breaker: If priority is identical, the newest email (higher id)
comes first
    return (a.id > b.id) ? 1 : 0;
}

// Swap two email objects in the heap

```

```

//inputs: two emails (email x and y)
//outputs: none (but swaps the values)
void swap(Email *x, Email *y) {
    Email temp = *x; //temporary value to keep the first email
    *x = *y;
    *y = temp;
}

// Create and initialize the heap
MaxHeap* create_heap(int capacity) {
    MaxHeap *heap = (MaxHeap*)malloc(sizeof(MaxHeap)); //malloc is for
memory for the heap
    heap->capacity = capacity; // how much it can hold
    heap->size = 0; // where it starts
    heap->arr = (Email*)malloc(capacity * sizeof(Email)); //keeps memory
fro the email capacity in our maxheap
    return heap;
}

// Shift up to maintain heap property
//input: the heap and the index out object to be heaped up is
//output: none, but shifts the objects in the heap iteratively to keep
everything in order
void heapify_up(MaxHeap *heap, int index) {
    while (index > 0) { //while its not at the root
        int parent = (index - 1) / 2; //email's parent in maxheap
        if (has_higher_priority(heap->arr[index], heap->arr[parent])) {
//if the child has more priority, switch
            swap(&heap->arr[index], &heap->arr[parent]);
            index = parent; //moves the index to the parent co check again
        } else {
            break; // done when the child isnt higher priority or reaches
root
        }
    }
}

//iterative version of heapify down, similar to heapify up
//inputs: the maxheap and the index

```

//output: no output, but properly swaps the higher priority child if there is one

```
void heapify_down(MaxHeap *heap, int index) {
    int size = heap->size;
    while (1) {
        int highest = index; //the index we are on i. e. parent
        int left = 2 * index + 1; //child (left)
        int right = 2 * index + 2; //child (right)
        //we check left then right child
        // checking the size helps not go off the array
        if (left < size && has_higher_priority(heap->arr[left],
heap->arr[highest])) {
            highest = left;
        }
        if (right < size && has_higher_priority(heap->arr[right],
heap->arr[highest])) {
            highest = right;
        }
        if (highest != index) { //if child is higher, we swap it
            swap(&heap->arr[index], &heap->arr[highest]);
            index = highest; // Step downward smoothly
        } else {
            break;
        }
    }
}

// Insert a new email item
void insert(MaxHeap *heap, Email email) {
    if (heap->size == heap->capacity) { //if the array's full, increase
the size
        heap->capacity *= 2;
        heap->arr = (Email*)realloc(heap->arr, heap->capacity *
sizeof(Email));
    }
    heap->arr[heap->size] = email; //add the email to the heap
    heapify_up(heap, heap->size); //we make sure the email is in the right
position in the heap
    heap->size++; //increase heap size
}
```

```

}

// Remove and return the maximum priority item
//input: the maxheap
//output: the removed email
Email extract_max(MaxHeap *heap) {
    Email max_email = heap->arr[0];
    heap->arr[0] = heap->arr[heap->size - 1];
    heap->size--; //reduce the size of the heap
    if (heap->size > 0) { //if the list has items, heapify down
        heapify_down(heap, 0);
    }
    return max_email;
}

// Deallocate heap arrays (garbage collection)
void free_heap(MaxHeap *heap) {
    free(heap->arr); //free email array
    free(heap); //free heap pointer
}

// Safely clean newline and carriage return characters from end of lines
//input: line
//no output
void trim_newline(char *str) {
    int len = strlen(str);
    while (len > 0 && (str[len - 1] == '\n' || str[len - 1] == '\r')) {
        str[len - 1] = '\0';
        len--;
    }
}

int main() { // the main processing part!
    char filename[MAX_STR_LEN]; //our variable for the file inputted
    printf("Enter input file name: ");
    if (scanf("%127s", filename) != 1) return 1; //gets filename, also
ends program in case of crash

    FILE *file = fopen(filename, "r"); //opens file (reads only, we dont
modify)

```

```

    if (!file) {
        printf("Error: Could not open file %s\n", filename);
        return 1;
    }

    MaxHeap *heap = create_heap(10); //create our starting heap
    char line[MAX_LINE_LEN];

    while (fgets(line, sizeof(line), file)) { //look through the lines of
the file line by line
        trim_newline(line); //gets rid of the extra stuff on the ends of
the line
        if (strlen(line) == 0) continue; //skips blank lines

        // Separate command text from arguments by targeting the first
space delimiter
        char line_copy[MAX_LINE_LEN];
        strcpy(line_copy, line); //creates a copy of the line in the list
to use

        char *cmd = strtok(line_copy, " ");
        if (cmd == NULL) continue; //skips if empty

        if (strcmp(cmd, "EMAIL") == 0) { //if the command is email,
            // Find the start of arguments past the command name text and
its space
            char *args_start = line + 6;

            char args_copy[MAX_LINE_LEN];
            strcpy(args_copy, args_start);

            // Tokenize strict comma boundaries (spaces inside subject
fields are left intact)
            char *cat = strtok(args_copy, ","); // get category
            char *subj = strtok(NULL, ","); // get subject
            char *dt = strtok(NULL, ","); // get date

            if (cat && subj && dt) { //make new email variable if the
above strings exist
                Email new_email;

```

```

        new_email.id = ++insertion_counter; //newness counter

        strcpy(new_email.category, cat); //we need to make copies
of whats in the pointers above
        strcpy(new_email.subject, subj);
        strcpy(new_email.date, dt);
        new_email.priority = get_priority(cat); // and the
priority

        insert(heap, new_email); //puts the email in the heap
    }
}
else if (strcmp(cmd, "NEXT") == 0) { //if the command is NEXT
    if (heap->size > 0) { //if theres stuff in the maxheap
        Email top = heap->arr[0]; //get the top email
        printf("[%s] Subject: %s | Date: %s\n", top.category,
top.subject, top.date); //print the info
    } else { //else tell user theres no emails
        printf("No emails in queue.\n");
    }
}
else if (strcmp(cmd, "READ") == 0) {
    if (heap->size > 0) { //if theres emails in the heap, remove
the top one

        Email read_email = extract_max(heap);
    } else {
        printf("No emails to read.\n");
    }
}
else if (strcmp(cmd, "COUNT") == 0) {
    printf("There are %d emails to read.\n", heap->size);
}

fclose(file); //closes the file
free_heap(heap); //garbage collects
return 0; //end of code.
}

```